

CUI

Artificial Intelligence Integration Center (AI2C) Container Software Design Document (SDD)

For
(Tenant 0)

Version v1.0.0



21 May 2025

Prepared for:

AI2C

CW3 Vic Fernandez III

victor.fernandez19.mil@army.mil

6425 Living Place, Pittsburgh 15206

CUI

Contents

Classification Instruction	2
Responsible Organization	2
Points of Contact	2
cDSO Container Technology Pipeline Process	3
Process	3
Software Overview	4
Purpose	4
Intended Users	4
Identification	4
Security Overview	5
System Categorization	5
Architecture Diagram	7
Detailed Software Description	8
Software Purpose / Goals	8
Software Functionality	8
Software Components	8
User Interface	8
Configuration Management	8
Development and Testing	8
Technical Overview	9
Networking Capabilities	9
Ports / Protocol / Services	9
Encryption	9
Authentication	9
Operating System	9
Role-Based Access Control (RBAC)	9
Audit Capabilities	9
Dependencies	9
Assumptions	9
Constraints	10
Scanning Tools & Remediation	10
Security Technical Implementation Guides (STIGs)	10
Appendix A – References	11
Appendix B – Acronyms	12
SRG (Security Requirements Guide)	13

Classification Instruction

(U) Add the portion marking before each paragraph. This paragraph is marked as Unclassified, as denoted by the '(U)' at the beginning of this paragraph.

Responsible Organization

(U) Who is the cognizant Authorizing Official (AO) for the organization?

Mr. Joseph Welch

(U) Is your system funded by Title 10 or Title 50?

Title 10	Title 50
x	

Points of Contact

Name	Role	Phone Number	Email
LTC Scott Anderson	Mission Owner	(978) 654-3210	scott.t.anderson.mil@army.mil
CPT Andre Michell	Information System Owner	(978) 654-3210	andre.j.michell.mil@army.mil
CW3 Victor Fernandez III	Information System Security Manager	(978) 654-3210	victor.fernandez19.mil@army.mil
CW3 Jilly Gonzalez	Information System Security Officer	(978) 654-3210	jilly.m.gonzalez.mil@army.mil

cDSO Container Technology Pipeline Process

(U) The cDSO is a low cost, secure, flexible, and enterprise-wide containerized application development pipeline. Using the cDSO improves the quality and timeliness of the Risk Management Framework (RMF) process. It distributes the responsibility for security while enabling developers at all levels and organizations to develop and secure containerized applications with unmatched agility.

Process

- a. (U) System Owner completes CSDD and BoE with Cybersecurity Office assistance
- b. (U) Cybersecurity Office POC reviews BoE and sends it to ISSM.
- c. (U) ISSM reviews BoE and sends it to SCA-V or returns for rework.
- d. (U) SCA-V reviews BoE and signs approval or returns for rework.
- e. (U) ISSM signs approval. ISSM enters approval in the GitLab Merge Request.
- f. (U) SCA-V enters approval in the GitLab Merge Request and Merges the CSDD.
- g. (U) Container Software/Pipeline can now be installed, and BoE is incorporated into System BoE

Software Overview

Purpose

(U) Tenant 0 uses Machine Learning to predict the species of an Iris flower based on the petal and sepal measurements given. Users can input measurements, select different classification models (like KNN, SVM, or Decision Tree), and visualize how each model makes predictions through dynamic 2D and 3D plots. Designed for students, educators, and data enthusiasts, Tenant 0 offers an intuitive way to explore ML concepts while classifying one of the most famous datasets in data science.

Intended Users

- a. (U) Administrators.
- b. (U) Users.

Identification

Clearly identify the type of software/pipeline to be used, all organizations that will use the software/pipeline, and the networks/domains on which the software/pipeline will be deployed

Software Name	<Application Name>
Version	Version

	Software Type
	Commercial Off-the-Shelf (COTS)
X	Government Off-the-Shelf (GOTS)
	Free and Open Source Software (FOSS)
	Contract-Developed / Modified Off-the-Shelf (MOTS)

	Architecture
	Installed on Desktop
X	Application Server with Database
	Application Server with Persistent Storage

	Users of the Software
X	U.S. Army Cyber Command
	U.S. Army Cyber Command and other DoD organizations

	Networks
	JWICS
	SIPRNET
X	NIPRNET
	STEMSNET
	Stand-Alone (Name:)

	Privacy Act Data
	"Business" or "Rolodex" PII
	PII
	PHI

Security Overview

System Categorization

Security Categorization is the first step in the Risk Management Framework (RMF) and paramount because of its effect on all other steps in the framework, from selection of security controls to the level of effort required in assessing security control effectiveness. The Security Categorization will generally be determined based on the most sensitive or critical information collected by, consumed by, received by, processed in, stored in, and/or generated by the system/application.

Security categorization of the system that is the consumer of this pipeline:

Confidentiality

- ☒ LOW
☐ MODERATE
☐ HIGH

Integrity

- ☒ LOW
☐ MODERATE
☐ HIGH

Availability

- ☒ LOW
☐ MODERATE
☐ HIGH

	Anticipated Domain(s) (CUI when filled out)
	Unclass (IL2) (DEV)
X	NIPRNet (IL5) (TEST, PROD)
	SIPRNet (DEV, TEST, PROD)
	JWICS (DEV, TEST, PROD)

	Anticipated Dissemination Controls (SIPR) (CUI when filled out)
	NOFORN
	FRD
	NATO
	ORCON
	PROPIN
	RESEN
	REL TO USA, AUS, CAN, GBR
	REL TO USA, AUS, CAN, GBR, NZL
	Other: (Please specify)

	Anticipated Dissemination Controls (JWICS) (CUI when filled out)
	SI-G
	TK
	HCS-P
	COMINT
	MASINT
	NATO
	REL TO USA, AUS, CAN, GBR
	REL TO USA, AUS, CAN, GBR, NZL
	COMSEC
	Other: (Please specify)

Architecture Diagram

Here you should attach an application architecture diagram that shows the composition of your application, and how all of its components interact with one another. Please include data flows between your applications, inputs from other systems, and outputs to other systems. Ensure that you cover security relevant components within your architecture diagram in below sections. Reminder that your repo will be copied into the container that will be generating your CSDD PDF, so you should reference all files relative to the top-level directory of your repository.

The name in the square brackets represents the figure name, while the path in the parentheses should be the image you want to include. Please ensure that the data flows are clearly identified, as this is required by eMASS.



Figure 1: Architecture Diagram

Detailed Software Description

Software Purpose / Goals

Give a detailed description of the mission-oriented purpose for the software. This section should not be overly technical in nature, but rather serves to give context for the gap the software is filling and should make a compelling case for it.

Software Functionality

Describe how the software is going about achieving the aforementioned goals.

Software Components

Describe each component of your software with a name, associated technology, and the function it serves. You can think of this section as accompanying the architecture diagram by effectively labelling each piece depicted therein.

User Interface

Provide a description and / or example (screenshots, comparisons to existing UIs, etc.) of your software's user interface. This might be a Graphical User Interface (GUI), a Textual User Interface (TUI), or even a Command-Line Interface (CLI). Describe how the user is expected to interact with the software. How does the user interface with the capability? Do they PKI log in, use MFA, etc?

Configuration Management

How the user is expected to manage configurations for your application. This could be via a configuration file (.ini, .toml, etc.), via environment variables, etc. Consider how users can manage multiple configurations flexibly and how users can rollback configurations.

Development and Testing

Explain in detail your development methodology, process, and testing stack. This should include any relevant project management facilitation software or techniques, as well as what testing framework you intend to use.

Milestones Explain how you are measuring progress for this application. These could be based on time, feature completion, or various other metrics. One approach might be to use the SMART framework, where you set Specific, Measurable, Achievable, Relevant and Time-bound goals.

Failure Conditions & Mitigations Define failure and success for this project, and determine what foreseeable circumstances would lead to failure, and what mitigations can be emplaced to counteract these conditions or lessen the impact of such a failure. What happens when your capability fails? What is the plan to get it back? Think COOP or IR.

Technical Overview

Networking Capabilities

Describe in detail what networking requirements and capabilities are needed and produced by this software.

Ports / Protocol / Services

Describe which ports, protocols, and services are used by this application. Describe under what conditions each will be used and the purpose for each. If not listed explicitly here, all other ports, protocols, and services will be assumed unneeded and thus unavailable in the application context.

Encryption

Describe what encryption standards will be used. Specify your software components that will be using them, what purpose they serve, and how they will be used.

Authentication

Describe what authentication scheme will be enforced and how you plan to do so.

Operating System

What operating systems will your software be built to run on. You should specify the OS family, version, and architecture that your system is built to run on. It is your responsibility to ensure that any OS listed here is supported through your testing systems as well.

Role-Based Access Control (RBAC)

List RBAC specifications and enforcement mechanisms.

Audit Capabilities

This application uses the Continuous Integration (CI) services provided through Community DevSecOps (CDSO), which is an inner-sourced CI solution provided for use within the Department of the Army. The pipelines provided by CDSO are all open and auditable for anybody within the department of the Army. The source code for all components can be found here: <https://code.code.cdso.army.mil/cdso>. The documentation can be found here: <https://cdso.pages.code.cdso.army.mil/>.

Dependencies

This application's dependencies can be viewed by looking at the pipeline artifacts for the most recent CI iteration under the `syft-grype` job artifacts.

This job produces a `syft.json` artifact which contains the Software Bill of Materials (SBOM) for the application image.

Assumptions

List any assumptions made during the design of this application. This could include assumptions about the network or operating environment, assumptions about the end-user, etc.

Constraints

List what constraints there are that affect this application in its design, implementation, deployment, etc.

Scanning Tools & Remediation

This application uses all of the scanning tools as defined in the `application-security-review` Pipeline from CDSO. You can find details about the pipeline and its tools here: <https://cdso.pages.code.cdso.army.mil/components/pipelines/#application-security-review.yml>.

Its definition can be found here: <https://code.code.cdso.army.mil/cdso/cdso/-/blob/main/pipelines/application-security-review.yml>.

Security Technical Implementation Guides (STIGs)

Appendix A – References

List relevant sources that were referenced throughout the rest of the CSDD.

Appendix B – Acronyms

Make sure to add any acronyms used in this CSDD that are pertinent to your organization and application/pipeline.

Acronym	Definition
CDSO	Community DevSecOps
CI / CD	Continuous Integration / Continuous Deployment

SRG (Security Requirements Guide)

Add the following: List the SRG that is applicable to the system these containers/pipelines support.